Automated Data Pipeline

Airline Passenger Satisfaction

PySpark

Apache Airflow

MongoDB Atlas

Great Expectations

P2M3 · Zafirah Aida Adista

Phase 2 · Milestone 3 · Data Engineering

Executive Summary

This project aims to design and implement an end-to-end automated ETL pipeline leveraging PySpark for scalable processing, Great Expectations for data validation, Airflow for orchestration, and MongoDB as the final data store.

Problems

- Manual data collection is slow & error-prone
- No standardised data quality checks in place
- Insights delayed due to lack of automation

Key Question

What is the best strategy for building automated, scalable data workflows that consistently ensure data integrity for downstream analytics?

Solutions

- Automated ETL pipeline with Apache Airflow
- PySpark for scalable data extraction & transformation
- Great Expectations for data quality validation
- Store data in MongoDB

Bottomline: This pipeline demonstrates a robust and automated ETL process integrating data validation, transformation, and orchestration.

Background & Motivation

Why automate airline passenger satisfaction analytics?

✈ Business Goal

The airline aims to improve customer retention and service quality by building data-driven insight on passenger satisfaction, delay patterns, and service ratings across different flight classes and routes.

Industry Context

Airlines operate in a highly competitive market. Understanding what drives passenger satisfaction is critical to retaining customers and improving service quality.

Data Challenge

Without a proper pipeline, passenger data arrives incomplete, poorly formatted, and spread across multiple sources, none of which are immediately usable for analysis

Engineering Need

A modern data engineering pipeline is required to reliably extract, clean, and store this data in a queryable format to support analytics and ML workflows.

Background

Workflow

Pre Automation

Dataset Preparation → Data Validation

Great Expectations

Data Pipeline Automation

Extract → Transform → Load

Airflow

Dataset Overview

Airline Passenger Satisfaction Survey – train.csv

Passenger Records : 103,594 | Fields : 25

Passenger Demographics

· Gender (Male / Female)
· Customer Type (Loyal / Disloyal)
· Age
· Id

Flight Information

· Class (Business / Eco / Eco Plus)
· Flight Distance
· Departure Delay in Minutes
· Type of Travel (Business / Personal)
· Arrival Delay in Minutes

Service Ratings (0–5)

· Inflight Wifi Service
· Seat Comfort
· Food & Drink
· Inflight Entertainment + 8 more...

Target Variable

· Satisfaction (satisfied / neutral or dissatisfied)

Great Expectations

*is used to validate and ensure data quality : **all 7 expectations passed***

1	**Column values 'id' must be unique**	id : Ensuring that each row of data represents a unique passenger entity and that no duplicate records exist within the system
2	**All ratings must be between 0 and 5**	Ratings : The rating scale used in this survey ranges from 0 to 5
3	**Column values 'Class' must be in set = ["Business", "Eco", "Eco Plus"]**	Class : Guarantees only valid label values enter the model training pipeline.
4	**Column values 'Flight Distance' must be in ['int'] type**	Flight Distance : Ensures numerical type integrity for aggregation and distance-based analyses.
5	**Column 'Gender' must consist of 2 categories**	Gender : Ensures it contains exactly 2 unique values (Male and Female)
6	**Maximum Delay values must be within operational limits**	Arrival & Departure Delay : Ensure that maximum recorded delays remain within reasonable operational limits (0 to 2000 minutes)
7	**The Age column must not contain negative values**	Age : Ensure logical passenger age data by setting a minimum threshold of 0 years

Required **Additional (Non-lecture)**

ETL Process

1 Extract

kaggle

- **Source:** Airline Passenger Satisfaction

1. SparkSession initialised
2. Read CSV with inferSchema
3. Save extracted CSV output

`extract.py`

2 Transform

Missing Values Handling

1 Reads extracted CSV via PySpark

2 Identifies nulls in 'Arrival Delay in Minutes' (310 nulls)

3 Computes column median with approxQuantile()

4 Fills missing values with median (na.fill)

5 Saves cleaned data as transform_result_*.csv

`transform.py`

3 Load

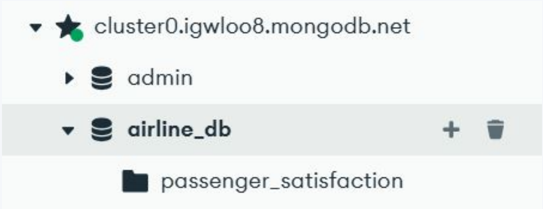
1 Reads transformed CSV via PySpark

2 Connects to MongoDB Atlas via pymongo

3 Converts Spark DF → Pandas → dict list

4 Inserts all records with insert_many()

5 Stored in airline_db.passenger_satisfaction



`load.py`

Automation Data Pipeline

DAG Overview

1 **#task_id**

"extract" >> "transform" >> "load"

2 **#start_date**

>> dt.datetime(2024, 11, 1) + timedelta(hours=7)

3 **#schedule**

crontab "10,20,30 2 * * 6"
>> every 10 min at 9:10–9:30 AM Saturday

4 **#max_active_runs**

1 (sequential, concurrency=1)

THANK YOU